Regular Expressions

Definitions

Equivalence to Finite Automata

RE's: Introduction

- *Regular expressions* are an algebraic way to describe languages.
- They describe exactly the regular languages.
- If E is a regular expression, then L(E) is the language it defines.
- We'll describe RE's and their languages recursively.

RE's: Definition

- Basis 1: If *a* is any symbol, then **a** is a RE, and L(**a**) = {a}.
  - Note: {a} is the language containing one string, and that string is of length 1.
- Basis 2: ϵ is a RE, and L(ϵ) = {ϵ}.
- Basis 3: $\varnothing$ is a RE, and L($\varnothing$) = $\varnothing$.

Example - $\sum = \{a, b\}$ and r is a regular expression of language made using these symbols

Regular language	Regular set
∅	{ }
∈	{∈}
a*	{∈, a, aa, aaa .....}
a+ b	{a, b}
a.b	{ab}
a* + ba	{∈, a, aa, aaa,..... , ba}

RE's: Definition – (2)

- Induction 1: If E_1 and E_2 are regular expressions, then E_1+E_2 is a regular expression, and $L(E_1+E_2) = L(E_1) \cup L(E_2)$.
- Induction 2: If E_1 and E_2 are regular expressions, then E_1E_2 is a regular expression, and $L(E_1E_2) = L(E_1)L(E_2)$.

Concatenation : the set of strings wx such that w Is in $L(E_1)$ and x is in $L(E_2)$.

RE's: Definition – (3)

- Induction 3: If E is a RE, then E* is a RE, and L(E*) = (L(E))*.

Closure, or "Kleene closure" = set of strings $w_1w_2\ldots w_n$, for some $n \geq 0$, where each w_i is in L(E).
Note: when n=0, the string is ϵ.

Precedence of Operators

- Parentheses may be used wherever needed to influence the grouping of operators.
- Order of precedence is * (highest), then concatenation, then + (lowest).

Examples: RE's

- L(**01**) = {01}.
- L(**01**+**0**) = {01, 0}.
- L(**0**(**1**+**0**)) = {01, 00}.
  - Note order of precedence of operators.
- L(**0***) = {ϵ, 0, 00, 000,… }.
- L((**0**+**10**)*(ϵ+**1**)) = all strings of 0's and 1's without two consecutive 1's.

Identities for Regular Expression

- There are many identities for the regular expression. Let p, q and r are regular expressions.
- $\emptyset + r = r$
- $\emptyset.r = r.\emptyset = \emptyset$
- $\in.r = r.\in = r$
- $\in^* = \in$ and $\emptyset^* = \in$
- $r + r = r$
- $r^*.r^* = r^*$
- $r.r^* = r^*.r = r^+$.
- $(r^*)^* = r^*$
- $\in + r.r^* = r^* = \in + r.r^*$
- $(p.q)^*.p = p.(q.p)^*$
- $(p + q)^* = (p^*.q^*)^* = (p^* + q^*)^*$
- $(p+ q).r = p.r + q.r$ and $r.(p+q) = r.p + r.q$

Prove that $(PQ)^* P = P(QP)^*$

Solution

Start with the left-hand side (LHS)

$(PQ)*P$

Expand $(PQ)^*$

$\{\Lambda, PQ, PQPQ, PQPQPQ, \ldots\}$ P

Concatenate P with each element

$\{P, PQP, PQPQP, PQPQPQP, \ldots\}$

Factor P,

$P\{\Lambda, QP, QPQP, QPQPQP, \ldots\}$

Recognize the set,

$P(QP)*$

Thus, LHS equals the right-hand side (RHS).

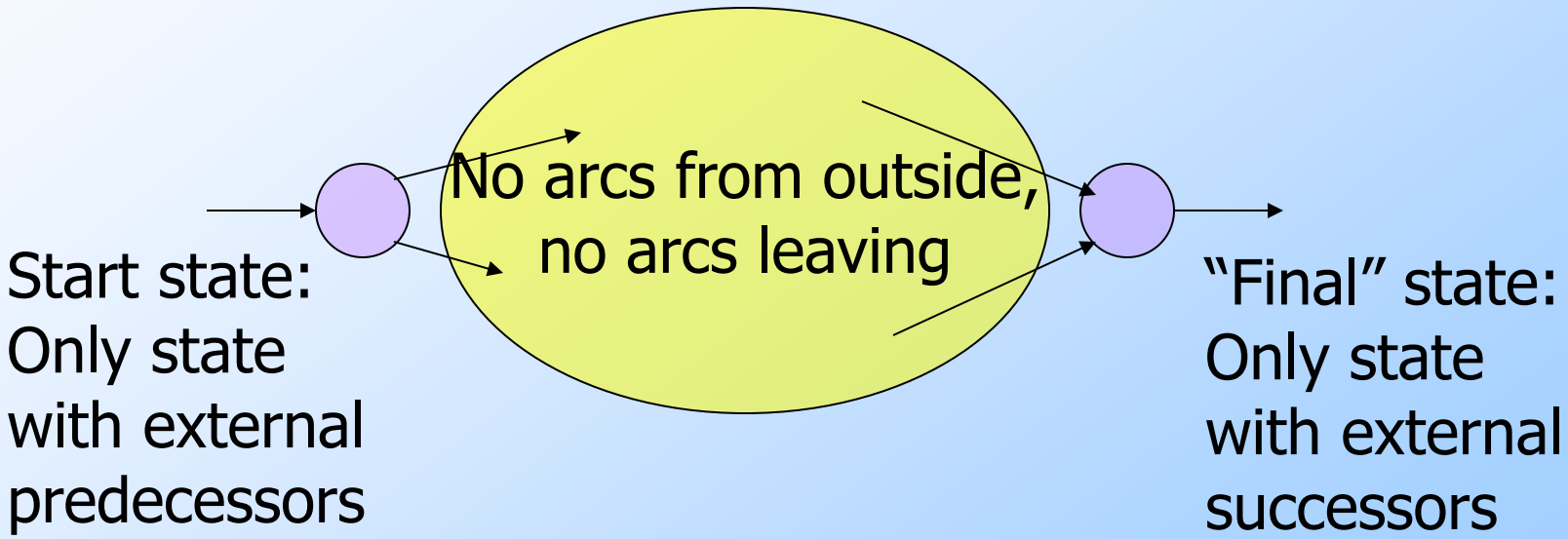
Equivalence of RE's and Automata

- We need to show that for every RE, there is an automaton that accepts the same language.
  - Pick the most powerful automaton type: the ϵ-NFA.
- And we need to show that for every automaton, there is a RE defining its language.
  - Pick the most restrictive type: the DFA.

Converting a RE to an ϵ-NFA

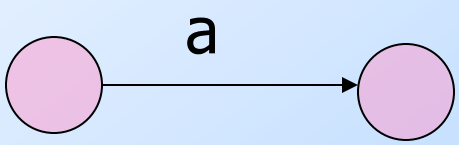
- Proof is an induction on the number of operators (+, concatenation, *) in the RE.
- We always construct an automaton of a special form (next slide).

Form of ϵ-NFA's Constructed

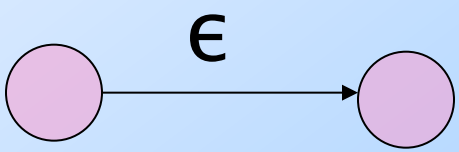


RE to ϵ-NFA: Basis

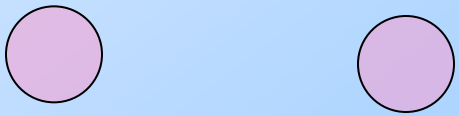
- Symbol **a**:



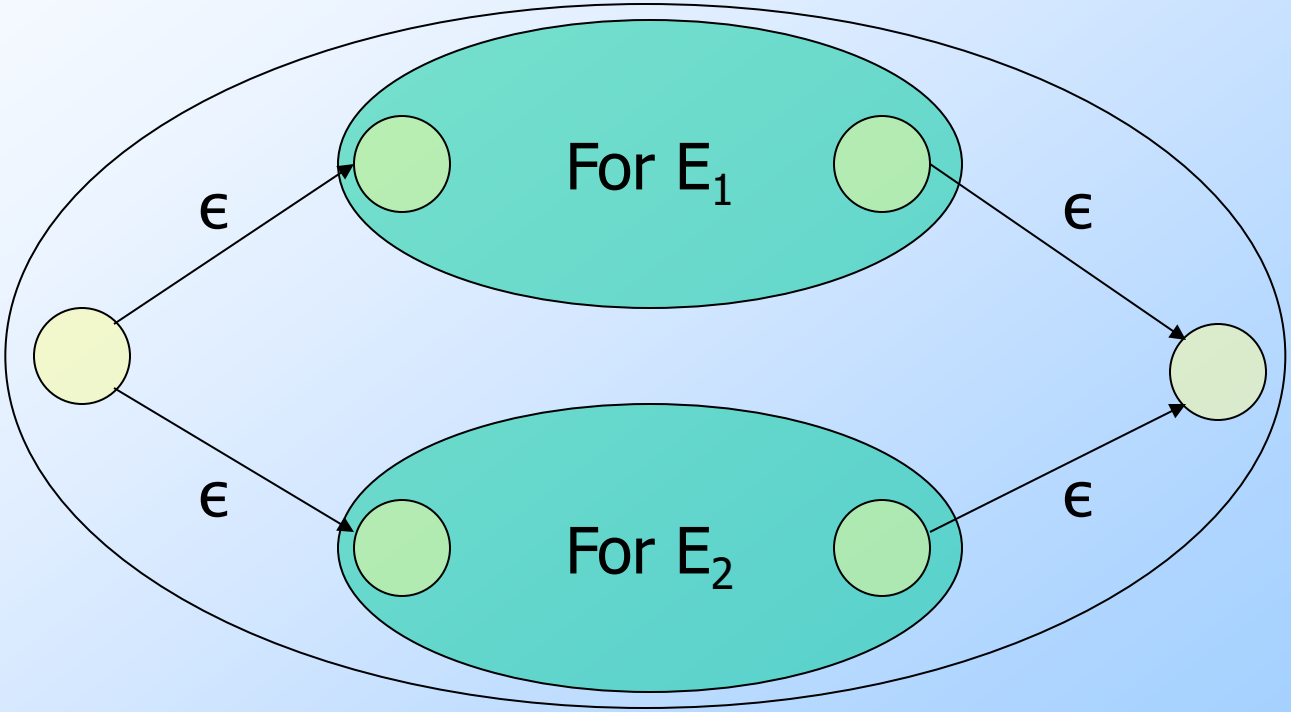
- ϵ:



- ∅:

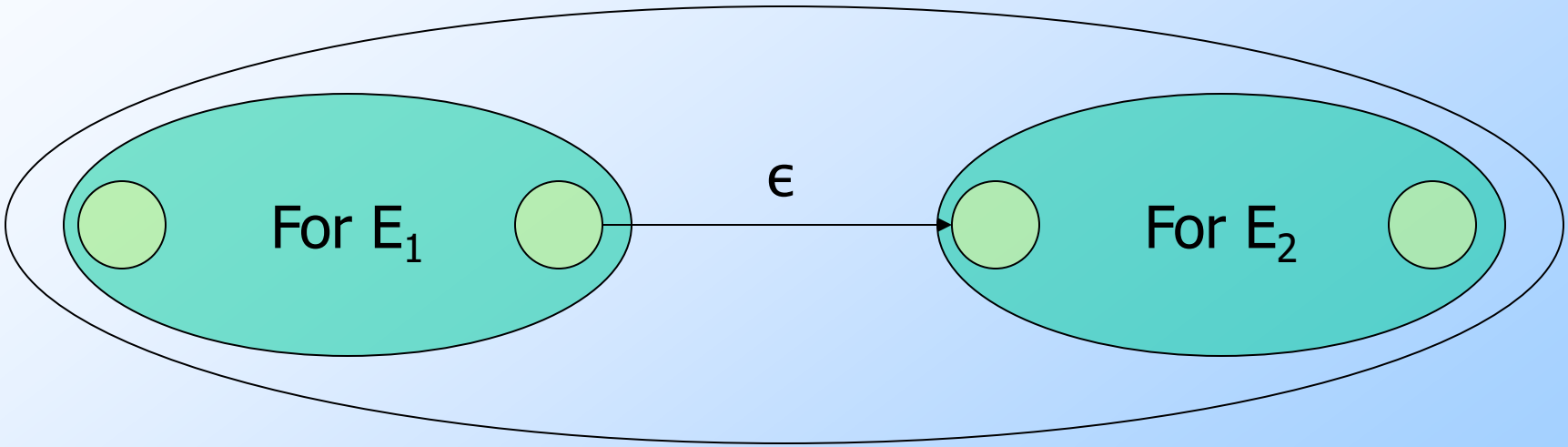


RE to ϵ-NFA: Induction 1 – Union



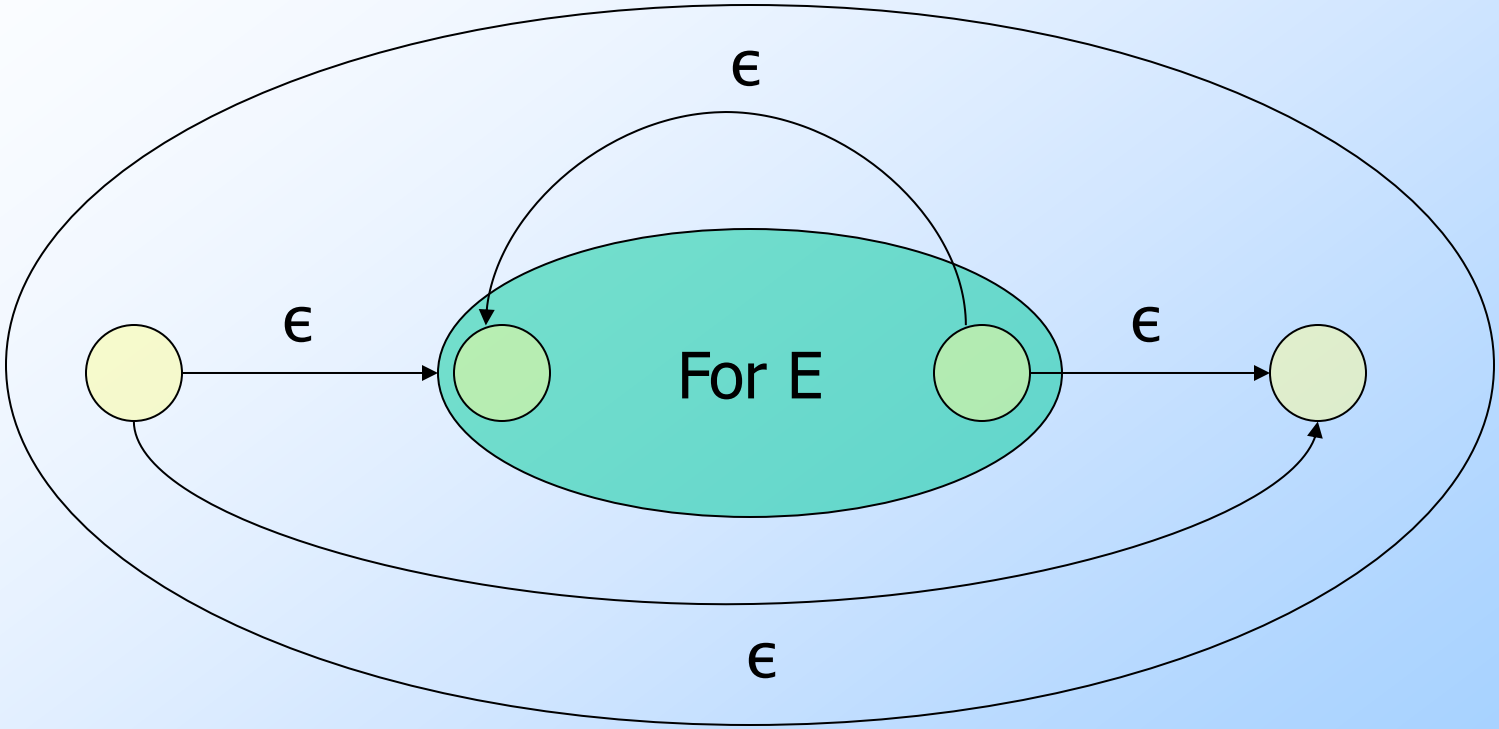
For $E_1 \cup E_2$

RE to ϵ-NFA: Induction 2 – Concatenation



For E_1E_2

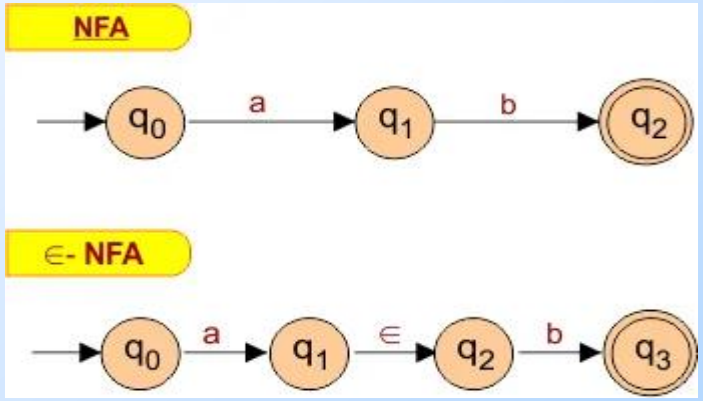
RE to ϵ-NFA: Induction 3 – Closure



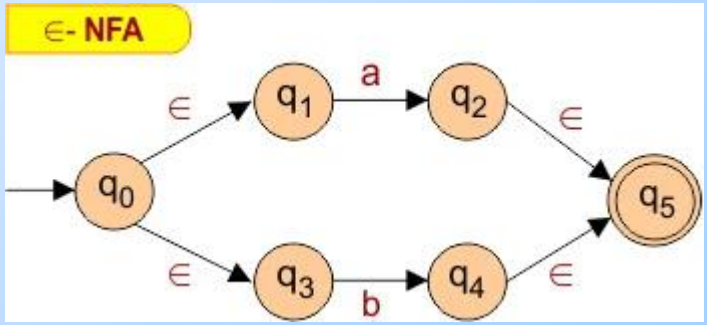
For E*

R.E to NFA With Epsilon by using Thomsons notation

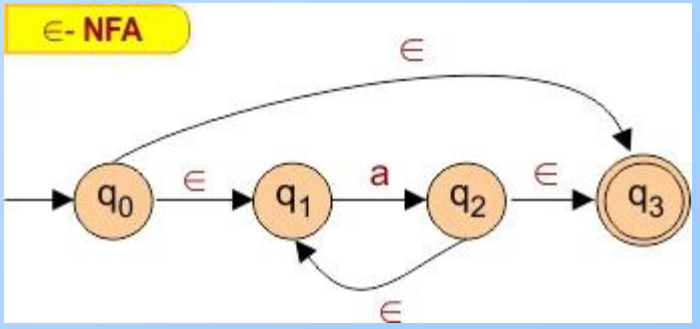
Create a ∈-NFA for regular expression: ab or a/b



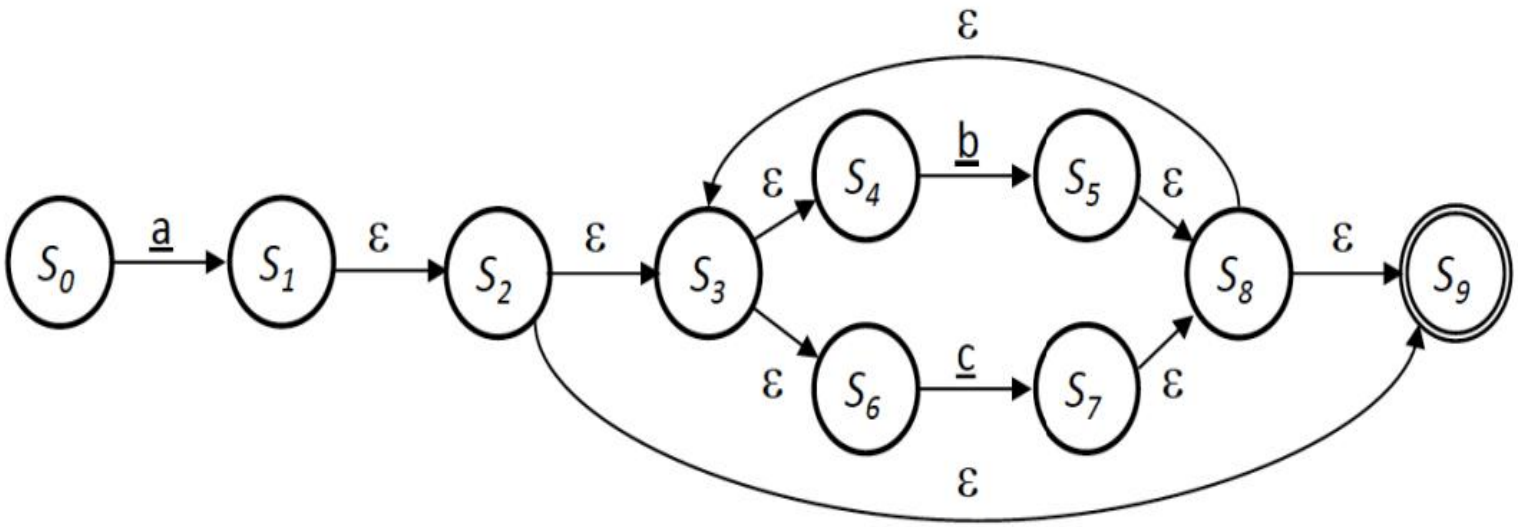
Create a ∈-NFA for regular expression: a+b



Create a ∈-NFA for regular expression: a*



For a(b|c)*



DFA-to-RE

- In order to find out a regular expression of a Finite Automaton, we use Ardens Theorem along with the properties of regular expressions.
- ***Statement*** –
- Let **P** and **Q** be two regular expressions.
- If **P** does not contain null string,
- then **R = Q + RP** has a unique solution that is **R = QP***

- **Proof** –
- R = Q + (Q + RP)P [After putting the value R = Q + RP]
- = Q + QP + RPP
- When we put the value of **R** recursively again and again, we get the following equation –
- $R = Q + QP + QP^2 + QP^3$..
- $R = Q (\varepsilon + P + P^2 + P^3 + .)$
- R = QP* [As P* represents (ε + P + P2 + P3 + .)]
- Hence, proved.

Summary

- Each of the three types of automata (DFA, NFA, ϵ-NFA) we discussed, and regular expressions as well, define exactly the same set of languages: the regular languages.

Algebraic Laws for RE's

- Union and concatenation behave sort of like addition and multiplication.
  - + is commutative and associative; concatenation is associative.
  - Concatenation distributes over +.
  - Exception: Concatenation is not commutative.

Identities and Annihilators

- $\emptyset$ is the identity for +.
  - $R + \emptyset = R$.
- ϵ is the identity for concatenation.
  - $\epsilon R = R\epsilon = R$.
- $\emptyset$ is the annihilator for concatenation.
  - $\emptyset R = R\emptyset = \emptyset$.

Automata Theory - **Kleene Closure** and **Positive Closure** which is **Kleene plus**, which are essential in understanding the behavior of regular languages.

- Kleene Closure in Automata
- Kleene Closure (*), or star closure gives an infinite set of possible strings of any possible length, including null string or epsilon (ε), over input values of sigma (Σ). It is also known as Kleene Star and denoted by Σ^*.
- The following is a **mathematical notation of Kleene Closure** –
- $\Sigma_* = \Sigma 0 \cup \Sigma 1 \cup \Sigma 2 \cup \ldots$
- Where Σ0 represents epsilon (ε) and gives an infinite language.

Example

- Let us see this through an example. Consider the Σ input value is (01). If we apply the Kleene Closure (∗) over Σ, it will give the following result –
- $\{01\}^* = (0 \cup 1)^* = \{\epsilon, 01, 11, 00, 010, 110, \ldots n\}$,

  where n is infinity
- Here in the result $(01)^*$ will give all possible combinations of "0" and "1" of any length including epsilon (ε). So, we can say Kleene Closure always gives an infinite language.

Properties of Kleene Closure

- The following list highlights some properties of Kleene Closure –
- Empty Set
- Single Character
- Set of Characters
- Single String
- Set of Strings
- Special Case

- Empty Set
- Applying Kleene Closure to the empty set
- Expression
- $\emptyset^* = \{\epsilon\}$
- Single Character
- Applying Kleene Closure to a single character
- Expression
- $\{a\}^* = \{\epsilon, a, aa, aaa, aaaa, \ldots\}$

- Set of Characters
- Applying Kleene Closure to a set of characters
- Expression
- $\{x,y,z\}^* = \{\epsilon, x, y, z, xx, xy, xz, yx, yy, yz, zx, zy, zz, xxx, xxy, \ldots\}$
- Single String
- Applying Kleene Closure to a single string
- Expression
- $\{ab\}^* = \{\epsilon, ab, abab, ababab, \ldots n\}$

- Set of Strings
- Applying Kleene Closure to a set of strings
- Expression
- $\{010\}*=\{\epsilon,01,0,0101,010,001,00,010101,01010,01001,0100,00101,0010,0001,000,\ldots\}$

- Special Case
- Applying Kleene Closure to a special case
- Expression
- $\{a*bc*\}=\{a*+b+c*\}=\{b,ab,aab,c,\ldots n\}$

Positive Closure in Automata

- Positive Closure is similar to Kleene Closure except it excludes epsilon (ε). It also gives an infinite set of all possible strings of any length excluding epsilon (ε) over input values of sigma (Σ) The following is the mathematical notation of Positive Closure:

$$\Sigma+=\Sigma*-\{\epsilon\}$$

- Example
- Consider the **Σ** input value is (01). After applying positive closure (+) over Σ input gives the following result –
- {01}+=(0∪1)+={01,11,00,010,110,…n}, where n is infinity
- The result $(01)^+$ will give all possible combinations of "0" and "1" of any length excluding epsilon (ε). So, we can say Positive Closure always gives an infinite language.

Properties of Plus Closure

- **properties of Plus Closure** –
  - Empty Set
  - Single Character
- **Empty Set**
- Applying Positive Closure to the empty set results in an empty set
- Expression : $\emptyset^{+}=\{\}$
- **Single Character**
- Applying Kleene Closure to a single character
- Expression : $\{a\}^{*}=\{a,aa,aaa,aaaa,\ldots\}$

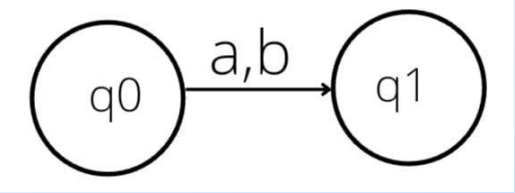
Difference between Σ^* and Σ^+

- The Σ^* contains the epsilon (ε) string along with other N strings, but $\boldsymbol{\Sigma^+}$ is just like $\boldsymbol{\Sigma^*}$ but does not hold the epsilon (ε) string. We can say –
- $\Sigma^* = \Sigma^+ + \Sigma^0$
- Σ^* Where Σ^0 represents epsilon (ε). So, holds the identity (single) element called Absalon, but Σ^+ does not have the identity element called epsilon (ε).

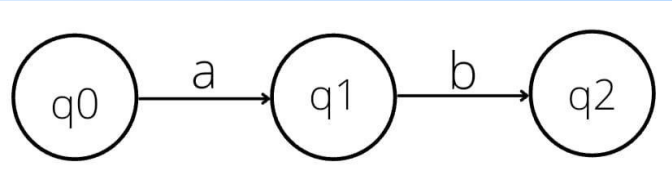
Steps To Convert Regular Expressions To Finite Automata

- To convert the RE to FA, the method that is popularly used is known as the **Subset method**. This method is used to get FA from the given regular expression.
- The steps in this method are given below:-
- **Step 1:** Make a transition diagram for a given regular expression, using NFA with ε moves.
- **Step 2:** Then, Convert this NFA with ε to NFA without ε.
- **Step 3:** Finally, Convert the obtained NFA to equivalent DFA.

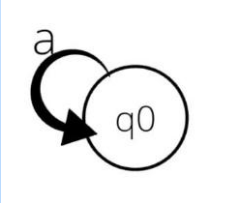
- Some standard rules help in the conversion of RE to NFA are:-
- 1.) If RE is in the form **a+b,** it can be represented as:

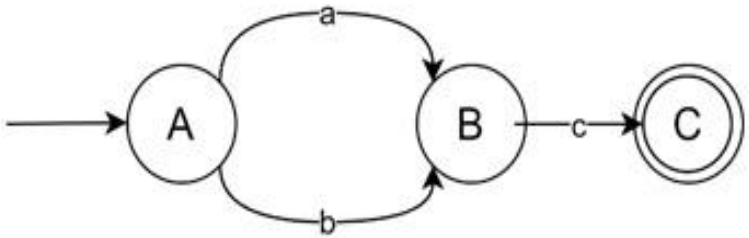


- 2.) If RE is in the form **ab,** it can be represented as:



- 3.) If RE is in the form of **a*,** it can be represented as:



Example:

R. E. =(a + b)c